LLM for Financial News Generation

Hugo Latendresse
17-644 - Applied Deep Learning - Spring 2025
Carnegie Mellon University

Abstract
This 17-644 Applied Deep Learning final project tackles the task of building and training a large language model (LLM) from scratch. The model is trained on the FNSPID dataset, which aggregates financial news articles, to answer the question of whether it is possible to train a transformer that completes sentences similar to those of a financial news article. We transformed and tokenized the FNSPID dataset in a way that can be ingested by a transformer, built a transformer architecture in PyTorch, and trained the transformer using a GPU and random search. In the end, we obtained a model with a perplexity of 19.69 that can complete sentences in a way that appears reasonable to a human and that sound like financial news.

Contents

1 Introduction

Financial news plays a crucial role in investment decision-making. The specific language, terminology, and structure of financial news articles represent a specialized domain of natural language. This project explores the application of a neural network to model and generate text in the style of financial news articles.

1.1 Problem Statement

We seek to answer the following question. Is it possible to build and train a transformer that completes sentences similar to those of a financial news article?

1.2 Data Source

The FNSPID dataset[1] consists of 10 million financial news articles in a csv. It is 22GB of unformatted, untokenized plain text. For example, the excerpt below is found in one of the articles:

"Agilent Technologies reported adjusted earnings of 404 million or 1.38 per share for the period. Analysts on average had expected the company to earn 1.34 per share, according to figures compiled by Thomson Reuters."

2 Exploratory Data Analysis

The dataset is one big table, saved in a single csv file. Since it is 22GB, we were not able to load it all in memory: we used a sample of 100,000 rows for the exploratory data analysis. Each row corresponds to one article.

There are fairly few fields in the dataset. For that reason, we were able to inspect them one by one. Below is a list of all fields, the percentage of time they are missing, and the action taken.

The date of publication of the article was helpful for the train/validation/test split: we did a temporal split, with the oldest articles in the train data to avoid leakage.

Article titles are valid text, but we focused on training the model to write the body of articles rather than their title, so we dropped that column.

We initially considered using the stock symbol (i.e., the stock ticker) associated with each article for the train/test/validation split, but decided that a temporal split is sufficient. Hence, we dropped the symbol field.

The URL of articles are not useful to our model, so we dropped that.

All 100,000 samples have Publisher and Author set to Null, so we ignored those fields.

The Article field is the key field of our dataset. It contains the text of the financial news article. This is the field we used to train our language model. The length of the articles in that field ranges from 1 character to over 200,000 characters, with a median of about 4,000 characters.

The remaining four fields are different types of summary of the main article. Those summaries were obtained by running automated algorithms that often created language very similar to the original text. This would not necessarily add value to our training, and we already have a gigantic dataset, so we ignored those fields.

Field Name	**Percentage Missing**	**Action**
Row Index	0%	Dropped
Publication Date	0%	Used for temporal split
Article Title	0%	Dropped
Stock Symbol	0%	Dropped
URL	0%	Dropped
Publisher	100%	Dropped
Author	100%	Dropped
Article	0%	Used for training
Lsa Summary	0%	Dropped
Luhn Summary	0%	Dropped
Textrank Summary	0%	Dropped
Lexrank Summary	0%	Dropped

3 Feature Engineering

No feature engineering was performed outside of the necessary step of transforming the data into tensors of tokens. The details of that are described in Appendix A. New features were not needed since the language model is able to learn patterns simply using existing sequences of text. We also purposefully did not use preprocessing such as removing special characters or converting upper case to lower case, since we want to generate those things.

4 Model Architecture

4.1 Transformer

We chose to implement a transformer architecture due to the ubiquity of such architectures in language modeling[2]. The specific architecture we built in this project is loosely inspired by Andrej Karpathy's Youtube series on neural networks[3] and by the Attention is All You Need paper[4].

A transformer consists of a series of "blocks". Each block has two components: one attention layer consisting of parallel self-attention heads, and one multi-layer-perceptron. The number of blocks is parametrized. Contrarily to the Attention Is All You Need paper, which was focused on machine translation, we implemented a decoder-only model (no encoder).

The first step was to define a multi-head attention module. We parametrized the number of heads to enable us to fine-tune that. We also parametrized the size of word embeddings. Since the task was next-word prediction, we used causal attention. That is, the prediction for one token did not have access to subsequent tokens in the sequence even if they are known during training, because they will not be known during inference.

We also defined a feedforward network ("FFN") module. Each FFN has only two layers, with one Gaussian Error Linear Unit ("GELU") activation function in between.

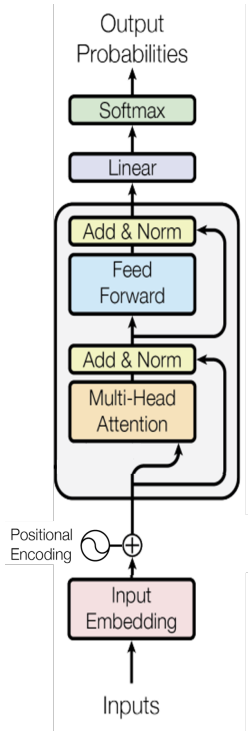

Figure 1: Summary of model architecture (modified version of a picture in the Attention Is All You Need[4] paper). This picture only shows one block (the large gray box). Our model has multiple such blocks.

We put together the attention module and the FFN to create one transformer block. We applied layer normalization to the input of each module. Moreover, the model has full residual connections, meaning that the input of every layer is the sum of the output of the previous

layer plus the (unaltered) input of the previous layer.

Putting everything together, our transformer is a stack of transformer blocks. Its forward pass goes as follows:
- Turn the token indexes into word embeddings.
- Add positional encodings (a representation of the position of each token in the sequence).
- Go through all transformer blocks, one after the other.
- Unembed by applying a linear layer to the final output to project back to the vocabulary space.
- Apply softmax.

Since next word prediction is fundamentally a classification problem, we used cross-entropy loss.

4.2 Custom Data Loader

To make our tokenized data accessible to the model during training, we created a custom data loader. Since we split our training tokens into chunks (see Appendix A), it was easier to build a custom data loader rather than using PyTorch's built-in data loader. Our data loader loads and switches to the next chunk once a given chunk is exhausted.

Moreover, prior to training, our tokens were simply saved as large one-dimensional NumPy arrays of integers. Our data loader converts those long lists into 2D PyTorch tensors based on a parametrized batch size and sequence length. Our transformer is able to ingest such tensors during training.

5 Training

5.1 Training on GPU

Since we are working with a training dataset of over two billion tokens (multiple gigabytes worth of plain text), it was unthinkable to try to run on CPU. We used the best GPU available on Google Colab, an NVIDIA A100 GPU. This GPU has a compute capability of 312 teraflops, which is about 5000 times that of the Intel i7 chip found on many laptops.

5.2 Model Performance Metrics

In previous homework, we typically showed the train and validation loss as well as the train and validation accuracy. For reasons explained in Appendix B, we decided to focus on only the validation loss this time.

Furthermore, we calculated the perplexity metric[5] on our test data but not on our validation data. We also explain why in Appendix B.

5.3 Training Rounds

We trained the model using the Adam optimizer.

The very first training round was a run of 1,000 training steps that was used to check whether there were any bugs in our architecture. The validation loss did decrease throughout those 1,000 steps, indicating that the architecture appeared to be correct.

Once we were confident that our model architecture was correct, we faced the challenge of performing a random search on a large dataset. On a smaller dataset, it is possible to perform a random search where each run "fully trains" the model on the entire dataset. However, in our case, the size of the dataset was prohibitively large. It would have been very long and expensive to fully train multiple versions of the model to see which hyperparameters work best. Instead, we first performed a random search with a wide range of hyperparameters in which the training of each

run stopped after 10,000 steps. The variable hyperparameters were: weight decay, number of attention heads per attention layer, number of transformer blocks, mini-batch size, embedding dimension, and learning rate. The result of that random search is given in Table 1. That first random search gave us a few insights:

- Attention head count should be at least 12. Both 12 and 16 worked well.
- Bigger batches perform better. This is not surprising, since it means each gradient update is more accurate. We can fix the batch size to 32.
- Embeddings should be at least 768. Both 768 and 1024 worked well.
- A smaller learning rate is better. This is not surprising, since the optimizer has a smaller chance of missing a minimum.
- This first experiment was inconclusive in terms of weight decay and block count.

Table 1: First random search (10,000 steps). Runs are sorted by validation loss.

Val. Loss	Wgt. Decay	Heads	Blocks	Batch	Emb. Dim	LR	Steps
3.69	0.05	12	6	32	768	0.00010	10,000
3.77	0.10	12	12	32	768	0.00010	10,000
3.79	0.05	16	12	32	1024	0.00010	10,000
3.83	0.10	12	6	32	768	0.00005	10,000
4.05	0.10	8	6	32	512	0.00010	10,000
4.52	0.10	12	12	8	768	0.00010	10,000
4.61	0.10	6	12	8	768	0.00010	10,000
6.73	0.05	12	6	32	768	0.00500	10,000

We used the insights acquired during the first random search to perform a second round of random search with fewer runs and narrower ranges of hyperparameters, but with 100,000 training steps. The results of that second random search are in Table 2.

Table 2: Second random search (100,000 steps). Runs are sorted by validation loss.

Val. Loss	Wgt. Decay	Heads	Blocks	Batch	Emb. Dim	LR	Steps
3.34	0.10	12	12	32	768	0.00001	100,000
3.35	0.05	12	6	32	768	0.00001	100,000
3.42	0.05	16	12	32	1024	0.00001	100,000
3.53	0.05	12	6	32	768	0.00005	100,000

Finally, we took the hyperparameters that worked best in the second random search (the first row of Table 2) and did a final run where we let the model train until the validation loss stopped improving. Our exact stopping criterion was to stop when the average validation loss of each of the last three slices of 10,000 training steps was worse than the average validation loss of the slice of 10,000 training steps before that. In other words, the model does not improve for 30,000 steps. It took 300,000 training steps to achieve that, as can be seen in Figure 2.

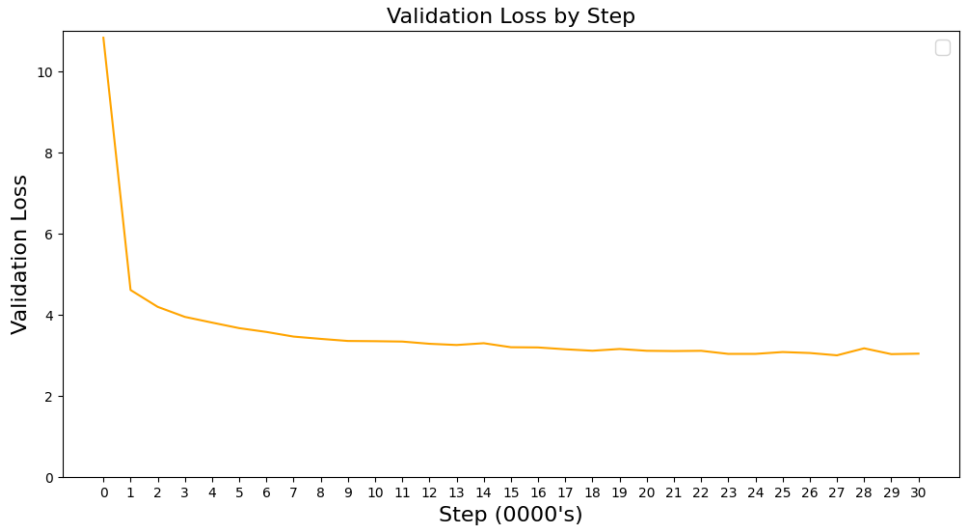

Figure 2: Training curve for final model. The validation losses after 280,000, 290,000, and 300,000 steps are worse than after 270,000 steps.

6 Results

6.1 Generation

To assess whether our final model could create coherent text, we asked the model to generate text by completing prompts like this one:

> Fintel reports that on December 13, 2023,

After 10,000 training steps, the model would use a vocabulary in line with financial news. However, it would make a lot of grammar mistakes and was not always coherent. Figure 3 shows an example.

Fintel reports that on December 13, 2023,000 shares of A shares, with an average price of $1.00 per share, which includes companies that are not the stock-to-based stock at a Zacks Rank #2 (Buy) stocks. The Zacks Consensus Estimate for the quarter is $1.00 to $3.00, with an average price of $2.30. The stock is expected to report its current price of $1.25 per share is scheduled to be a Zacks Rank #3 (Hold).

Figure 3: Generation after 10,000 training steps. Places where the model made poor next-word prediction decisions are highlighted in yellow.

The final model was different. In our humble opinion, it created text that is indistinguishable from real financial news. All sentences had a proper syntax and were coherent. Figure 4 shows an example.

Fintel reports that on December 13, 2023, the company reported a quarterly net loss of $0.02 per share on $1.02 billion in revenue. This compares to the year-ago quarter when earnings came to $0.04 per share on $1.03 billion in revenue.

What to watch: Shares of Zoom Video Communications are still down about 30% from their all-time high reached on February 12.

Figure 4: Generation after 300,000 training steps. The quality of model outputs is good.

6.2 Test Perplexity

Using our final model, we calculated performance metrics on the test dataset and obtained a test loss of 2.98 (exactly in line with the validation loss) and a test perplexity of 19.69. We were unable to find language modeling benchmarks for the FNSPID dataset, so we used benchmarks for another dataset,

Table 3: Comparison of Test Perplexity With Open Source Models

Model Name	Year	Dataset	Test Perplexity	Parameters (M)
Hybrid H3	2022	WikiText-103	10.60	2700
Megatron-LM	2019	WikiText-103	10.81	8300
GPT-2	2019	WikiText-103	17.48	1542
Our Model	2025	FNSPID	19.69	124

WikiText-103, which contains Wikipedia articles. Table 3 contains test perplexity scores of famous open source models tested on the WikiText-103 dataset [6] as well as the perplexity score of our model on the test portion of FNSPID.

Considering that our final model has 124M parameters, which is one order of magnitude smaller than the other models displayed in Table 3, our results may seem very close to GPT-2, which would be surprisingly good. This is most likely explained by the fact that financial news are fairly homogeneous in style and vocabulary. It makes it much easier for a model to achieve low perplexity compared to a more heterogeneous domain, such as Wikipedia.

7 Conclusion

In summary, we were able to build and train an LLM to generate text similar to financial news. An attention-based transformer architecture, a custom data loader, a GPU, and a multi-layered random search all contributed to successfully training an LLM on gigabytes of plain text.

Our hyperparameter optimization brought about some insight into model design for financial text generation. Models with at least 12 attention heads, embedding dimensions of 768 or higher, larger batch sizes, and lower learning rates consistently demonstrated superior performance.

In terms of generation quality, it was interesting to see the striking difference between a model trained for 10,000 steps and a model trained for 300,000 steps, despite their validation loss appearing to be close to each other (3.69 and 2.98, respectively). The model with a 3.69 loss made several obvious mistakes while the model with a 2.98 loss made no mistakes. This shows the importance of fully training a model until the validation loss stops improving, even when the dataset is large.

The fact that we could obtain a perplexity score on FNSPID similar to the perplexity score OpenAI's GPT-2 obtained on a more heterogeneous dataset, despite our model being much smaller, shows how the domain greatly impacts the next-word prediction task. Our domain was relatively narrow and allowed us to obtain good performance. On a more heterogeneous dataset, it would have been much harder to replicate the performance of GPT-2, whose training involved orders of magnitude more compute resources and trainable parameters than this project. We can conclude from this that training a good domain-specific model can be easier than training a good generic model. In terms of compute and model size, there are potential savings to be made when a domain-specific model is needed.

References

[1] Zihan1004. FNSPID. Hugging Face, 2023. URL https://huggingface.co/datasets/Zihan1004/FNSPID/tree/main/Stock_news.

[2] Brown et al. GPT-3: Language Models are Few-Shot Learners. *arXiv preprint arXiv:2005.14165*, 2020. URL https://doi.org/10.48550/arXiv.2005.14165.

[3] Andrej Karpathy. Neural Networks: Zero to Hero. YouTube Playlist, 2023. URL https://youtube.com/playlist?list=PLAqhIrjkxbuWI23v9cThsA9GvCAUhRvKZ.

[4] Vaswani et al. Attention Is All You Need. *arXiv preprint*, 2017. URL https://arxiv.org/abs/1706.03762.

[5] Wikipedia contributors. Perplexity - Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Perplexity, 2025. Accessed: 2025-04-23.

[6] Papers with Code contributors. Language Modelling on WikiText-103. https://paperswithcode.com/sota/language-modelling-on-wikitext-103, 2025. Accessed: 2025-04-23.

Appendix

A Data Transformations

The goal of our data transformations was to convert plain text into tokenized data that can be ingested by a model and split that into train, validation, and test datasets.

A.1 Data Clean-Up

As decided during the Exploratory Data Analysis, we loaded only two columns from the full (22GB) dataset: Date, used for the train/validation/test split, and Article, used as text data for training.

We deleted any row where one of the following two conditions is true:

1. Any of the two columns of interest is missing.

2. The length of the Article field is less than 100 characters.

The character threshold was selected to be 100 because anything less than that would be too short to contain financial news information and would probably be something like an error message, a disclaimer, or some other type of brief message we want to ignore.

The next step was to split our millions of news articles into training, validation, and test sets.

A.2 Train-Validation-Test Split

For timestamped data, it is always good to have held-out data be as of a later date than the training data. This way, there cannot be leakage of information from the held-out sets to the training set.

We split the rows according to the following rule:
- Articles published in 2021 and before are in the training dataset.
- Articles published in 2022 are in the validation dataset.
- Articles published in 2023 are in the test dataset.

A.3 Converting Text to Tokens

Our model needs a tokenizer to convert text into tokens and vice versa. Using simple concatenation, we first converted each dataset to a long string of text. Next, we applied the tokenizer to convert those long strings into lists of integers (each integer representing a token). Since this is a very compute-intensive process for such a large dataset, we did the following things:
- Break-up the text into chunks. Each chunk contains 100MB worth of text.
- Repeatedly save and load from file to avoid having to run again if it crashes.

To tokenize, we used the freely available "tiktoken" tokenizer library, which implements the tokenizer used by OpenAI for GPT-2. We fully converted the training set into chunks of tokens. The more training data we have, the better our model might become. The same is not necessarily true for the validation and test datasets. Once they are big enough, there is little to be gained from using even more tokens to calculate hold-out metrics. We just need samples that are a good representation of the population. Hence, to save on runtime, we only tokenized one chunk of the validation dataset and one chunk of the test dataset. Those would still represent 100MB worth of plain text each, which is huge. In the end, we have the following:
- The training data contains 82 chunks, for a total of 2,759,656,441 tokens.
- The validation data contains 1 chunk, for a total of 22,679,859 tokens.
- The test data contains 1 chunk, for a total of 22,637,887 tokens.

B Model Performance Metrics

B.1 Loss and Accuracy

In previous homework, we typically showed the train and validation loss as well as the train and validation accuracy. For the reasons explained below, we decided to focus on only the validation loss this time.

We don't show accuracy because language modeling is essentially a classification problem with about 50,000 classes (there are over 50,000 tokens to choose from in our vocabulary). A state of the art, large scale LLM may be able to predict exactly what the next token will be in its validation data a good percentage of the time, but we can't hope to achieve that with the limited compute resources we have. Our "small scale" LLM might be able to generate plausible-sounding text, but in terms of accurately predicting the exact next word in a text, it will be wrong the vast majority of the time. Hence, accuracy is not a very relevant metric here. Entropy loss, however, is still a relevant metric: it describes how "surprised" the model is from seeing each correct next token.

We don't show training loss because it would not add much relevant information to the chart. Typically, the model iterates over the training dataset over epochs. However, here, we are working with billions of tokens in the training set. With the set of hyperparameters we used, even 100,000 gradient steps uses only a fraction of the entire training set (i.e., it does not even correspond to one full epoch!). Hence, the training loss and validation loss are essentially the same thing, because in both cases, they reflect performance on data that the model has never seen before (again, since the model does not see the same training example twice). Since training and validation loss are essentially duplicative in this situation, we only show validation loss.

B.2 Perplexity

We also calculated the perplexity metric[5] on our test data, which measures the "surprise" of a model on test data. This metric is commonly discussed when talking about language model performance. We used it to compare the test performance of our model to open-source models.

Because perplexity is simply defined as exponential of cross-entropy loss, it is monotonic with respect to cross-entropy loss. In terms of validation metric, it would be redundant to look at both metrics to compare model runs, and so we focused on cross-entropy loss only during training. We only calculated perplexity on the test data using our final model. We chose to focus only on loss instead of only on perplexity simply because loss is on a smaller scale and therefore is easier to plot, look at, and compare. The loss of our models ranged from 0 to 11 whereas the perplexity would range from 0 to the exponential of 11 , which is over 50,000.